Informe de Cambios, Validación y Entregables

Desarrollador / PM: Jocelyn
Rol: PM y Backend Developer
Período de Trabajo: 23 de octubre al 26 de octubre
Área de Enfoque: Coordinación, desarrollo de robustez, límites de la cola, observabilidad y operación.

Resumen Ejecutivo

Durante este período de desarrollo se consolidaron y validaron los requisitos funcionales (RF) y no funcionales (RNF) correspondientes a los avances 2.1, 2.3, 2.4 y 2.5. La labor incluyó la verificación de mecanismos previamente implementados, la creación de documentación técnica clave y la refactorización e implementación de mejoras en el manejo de errores, trazabilidad de bitácora y estado de salud del sistema.

Detalle de Avances y Requisitos

Avance 2.1: Cola y Concurrencia (#9)

- RF-25 (#53) Rechazo explícito cuando la cola alcance capacidad:
- **Estado:** Previamente implementado / Verificado.
- **Descripción:** Se validó que al intentar encolar o ejecutar un trabajo cuando los límites de ejecución/cola están al máximo, el sistema responde explícitamente con un error de capacidad (`QueueFullError`).

- RNF-29 (#54) Aplicación de contrapresión ante límites de cola/memoria/descriptores:
- **Estado:** Previamente implementado / Verificado.
- **Descripción:** Se confirmó la correcta restricción del flujo de trabajo y la detención inmediata de la recepción de nuevas solicitudes ejecutables sin degradar ni sobrecargar los recursos del servidor/daemon.

Avance 2.3: Cancelación y Condiciones de Conflicto (#11)

- RF-27 (#64) Definición semántica y manejo de solicitudes duplicadas/reenviadas:

- Estado: Implementado y Documentado.
- **Acciones Realizadas**:Se redactó e integró la documentación oficial en el repositorio (docs/semantica-duplicados.md), estableciendo las reglas deterministas para la idempotencia, reintento de solicitudes y manejo de duplicados en la capa IPC.

Avance 2.4: Observabilidad y Robustez (#12)

- RF-14 (#66) Bitácora de eventos y errores con marca temporal e ID de trabajo:

- **Estado:** Implementado / Reforzado.
- **Acciones Realizadas:** Formateo centralizado en `src/daemon/logger.py` para asegurar que cada evento registrado en `jobsy.log` incluya la marca de tiempo UTC con milisegundos, nivel de evento y el identificador único del trabajo (`[Job: <UUID>]`).

- RF-23 (#67) Rechazo y registro de solicitudes que excedan límites:

- **Estado:** Implementado.
- **Acciones Realizadas:** Integración de trazas en la bitácora cuando se generan rechazos por saturación de capacidad (`QueueFullError`), registrando la causa explícita en los logs del servidor.

- RF-28 (#68) Conservación de estado coherente ante desconexión de clientes:

- **Estado:**Implementado.
- **Acciones Realizadas:** Manejo de excepciones de socket (`BrokenPipeError`, `ConnectionResetError`) en `src/daemon/server.py` al intentar enviar respuestas. La desconexión del cliente no altera ni corrompe el estado interno de los trabajos en memoria ni en disco.

- RNF-08 (#69) Resiliencia del servicio ante solicitudes inválidas o desconexiones:

- **Estado:** Previamente implementado / Verificado.
- **Descripción:** Comprobación de que ante payloads JSON malformados, acciones desconocidas o desconexiones abruptas, el loop del servidor se mantiene activo sin caerse.

- RNF-27 (#70) Garantía de transiciones de estado consistentes:

- **Estado:** Previamente implementado / Verificado.
- **Descripción:** Verificación de que los estados de los trabajos (`QUEUED`, `RUNNING`, `SUCCEEDED`, `FAILED`, `CANCELED`) sigan una máquina de estados estrictamente válida e irreversible hacia atrás.

Avance 2.5: Configuración, Operación y Calidad Local (#13)

- RF-24 (#73) Resumen de salud del servicio (trabajos activos, en cola, errores):

- **Estado:** Implementado.
- **Acciones Realizadas:** Se añadió la acción IPC `health` en `server.py`, el método `get_health_summary()` en `job_manager.py` y el subcomando `jobsy health` en `cli.py`, mostrando métricas en tiempo real de trabajos activos, en cola y fallas recientes.

- RNF-21 (#81) Mensajes de error claros con causa y acción sugerida:

- **Estado:** Implementado y Refactorizado.
- **Acciones Realizadas:** Estandarización de las respuestas de error en formato JSON mediante `make_error_response` (operación, causa y sugerencia) y refactorización limpia del CLI (`cli.py`) en `handle_error_response` para renderizar el mensaje estructurado de manera directa.

- RNF-22 (#82) Correlación en bitácora entre solicitud y trabajo:

- **Estado:** Implementado.
- **Acciones Realizadas:** Introducción de un identificador de solicitud `[ReqID: <hash>]` generado al recibir cada conexión IPC en `server.py`, permitiendo vincular en `jobsy.log` el flujo completo de una petición desde su recepción hasta su resolución o rechazo, aun antes de tener un `Job ID`.

- RNF-23 (#83) Guía de Usuario actualizada, seguimiento de entregables, AI-LOG, Registro de Riesgos e Issues/Cronograma:

- **Estado**: Documentación subida y actualizada.
- **Acciones Realizadas:** Se subieron las transcripciones de reuniones hasta el momento, se actualizó la guía de usuario hecha por Benjamin, se actualizó y mejoró el formato del readme, registro de riesgos posibles para el avance final.

Principales archivos Modificados e Integrados

1. **src/daemon/server.py:** Incorporación de `ReqID` para RNF-22, captura de errores con RNF-21 y endpoint de salud para RF-24.
2. **src/client/cli.py:** Refactorización de `handle_error_response` para RNF-21 y subcomando `health` para RF-24.
3. **src/daemon/logger.py:** Estandarización del formato de bitácora para RF-14 y RNF-22.

4. **src/daemon/job_manager.py:** Implementación del resumen de salud (`get_health_summary()`), manejo de transiciones de estado y recolección de métricas.
5. **docs/semantica-duplicados.md:** Documentación técnica de reglas de duplicados para RF-27.

Comandos Operativos y de Verificación Incorporados

A continuación se detallan los comandos clave agregados y utilizados para la operación y verificación del sistema:

1. Resumen de Salud del Servicio (RF-24)

Para obtener el estado global del daemon en tiempo real (trabajos activos, en cola, almacenamiento y fallas recientes):
```bash
**jobsy health**
```

2. Inspección de la Bitácora Centralizada (RF-14 y RNF-22)

Para visualizar en tiempo real los registros formateados con timestamps UTC, niveles de log, `Job ID` y el nuevo `ReqID` de correlación:
```bash
**tail -f ~/.local/state/jobsy/jobsy.log**
```

3. Validación de Mensajes de Error Estructurados (RNF-21)

Para comprobar las respuestas con causa y sugerencia en casos de IDs inexistentes o filtrado inválido:

```bash
# Error por ID de trabajo inexistente
**jobsy status 00000000-0000-0000-0000-000000000000**
**jobsy cancel 00000000-0000-0000-0000-000000000000**
**jobsy output 00000000-0000-0000-0000-000000000000**

# Error por argumento o filtro de estado inválido
**jobsy list --status INVALIDO**
```

4. Prueba de Capacidad y Contrapresión (RF-25, RF-23, RNF-29)

Para forzar la saturación de la cola y verificar el rechazo explícito tanto en la CLI como en la bitácora:

```bash
**execute sleep 30**
**execute sleep 30**
**execute ls**  # Deberá ser rechazado por capacidad si el límite configurado se ha alcanzado
```